

Multiprocessor CPU Scheduling Simulator

A Simulation of Scheduling Algorithms on Multiprocessor Operating Systems

Operating System Project Report

June 19, 2026

1 Introduction

1.1 General Overview

CPU scheduling determines which ready process is assigned to a processor and when that process is allowed to run. On a single-processor system, the scheduler selects work for one CPU. On a multiprocessor system, it must additionally decide how work should be distributed across several CPUs, whether processes should remain associated with a CPU, and when work should be migrated from an overloaded CPU to an idle or lightly loaded CPU.

Scheduling policies optimize different objectives. First-In, First-Out (FIFO) is simple and predictable, but a long process can delay all shorter processes behind it. Round Robin (RR) improves responsiveness by limiting each execution interval to a time quantum, although frequent switching can increase overhead. Multiprocessor schedulers must also consider load balance, processor affinity, local versus global queues, and migration cost.

This project implements a simulation environment for comparing several multiprocessor CPU scheduling strategies. Instead of executing real operating-system processes, it models processes with arrival time, CPU burst time, priority, and remaining execution time. Each algorithm produces a schedule that can be analyzed through metrics, Gantt charts, and queue-state animations.

1.2 Problem Statement

The project addresses the following problem:

Given a set of processes with different arrival times and CPU burst times, how can those processes be scheduled across multiple CPUs while maintaining good processor utilization, throughput, responsiveness, and load distribution?

The difficulty is that no single scheduling policy is optimal for every workload. A global queue is simple and naturally shares work among CPUs, but it can become a centralized scheduling point. Per-CPU queues improve locality and scalability, but they can become unbalanced. CPU affinity reduces process movement, while work stealing and explicit load balancing improve utilization by moving waiting work between CPUs.

The program therefore provides a common simulator that:

- Executes the same workload with multiple scheduling algorithms.
- Supports both global and per-CPU queues.
- Models preemptive and non-preemptive execution.
- Models process migration and migration overhead.
- Records process execution and queue state over time.
- Compares algorithms using consistent metrics and visualizations.

1.3 Project Objectives

The main objectives are:

1. Model processes and multiprocessor scheduling in Python.
2. Implement six scheduling approaches:
 - Global FIFO.
 - Global Round Robin.
 - Partitioned FIFO.
 - CPU Affinity.
 - Work Stealing.
 - Dynamic Load Balancing.
3. Compare their behavior under the same workload.
4. Visualize CPU execution and queue state.
5. Provide an interactive graphical interface for editing workloads and scheduler parameters.

2 Main Content

2.1 Process and System Model

Each simulated process contains:

Field	Meaning
id	Internal numeric identifier assigned by the program
burst_time	Total CPU time required by the process
arrival_time	Simulation time at which the process becomes ready
priority	Positive priority value stored with the process
remaining_time	CPU time still required during simulation

The current algorithms primarily use arrival time, burst time, and remaining time. Priority is validated and stored but is not currently used to choose the next process.

The system supports between 2 and 8 CPUs through the graphical interface and up to 100 processes in the `Processes` collection. CPU identifiers are zero-based internally and displayed as CPU 1, CPU 2, and so on.

Every scheduler produces:

```
steps: dict[int, list[ScheduleStep]]
```

Each `ScheduleStep` records:

```
process_id, begin_time, end_time, cpu_id
```

Schedulers also record a `history` containing running processes and waiting queues at scheduling decision points. This history is used by the dynamic visualization.

2.2 Scheduling Algorithms

2.2.1 Global FIFO (GLB_FIFO)

Global FIFO uses one shared ready queue for all CPUs. Processes enter the queue when their arrival time is reached. An available CPU removes the process at the front of the queue and runs it until completion.

Main characteristics:

- One global FIFO queue.
- Non-preemptive execution.
- Any available CPU can run the next process.
- Simple dispatch behavior.
- Long processes can cause a convoy effect for processes behind them.

The global queue naturally allows an idle CPU to obtain waiting work without explicit migration logic.

2.2.2 Global Round Robin (GLB_RR)

Global Round Robin also uses one shared ready queue. Unlike FIFO, a process runs for at most one time quantum:

```
run_time = min(time_quantum, remaining_time)
```

If the process does not finish, it returns to the end of the global queue.

Main characteristics:

- One global ready queue.
- Preemptive time-sliced execution.
- Better response time than non-preemptive FIFO.
- The quantum controls the balance between responsiveness and switching frequency.
- A very small quantum creates many schedule intervals, while a large quantum approaches FIFO behavior.

2.2.3 Partitioned FIFO (PAR_FIFO)

Partitioned FIFO maintains one FIFO queue per CPU. When a process arrives, it is assigned to the CPU with the smallest current workload. The workload includes the remaining time of the running process and all queued processes.

After assignment, a process stays on that CPU and runs to completion when it reaches the front of the local queue.

Main characteristics:

- One local queue per CPU.
- Non-preemptive FIFO execution.
- Initial load-aware placement.
- No migration after placement.
- Better queue locality than a global queue.
- A poor early placement cannot be corrected later.

2.2.4 CPU Affinity (CPU_Affinity)

CPU Affinity uses a global ready queue and a time quantum. The first time a process executes, it is mapped to a CPU. With hard affinity enabled, subsequent execution intervals of that process must occur on the same CPU.

Main characteristics:

- Global waiting queue.
- Preemptive Round Robin execution.
- Process-to-CPU mapping.
- Hard affinity preserves CPU placement.
- Affinity can represent cache-locality benefits.
- Strict affinity may leave a CPU idle even when runnable work exists for another CPU.

The implementation also supports soft affinity through the `hard=False` option, although the dashboard currently creates the scheduler with hard affinity enabled.

2.2.5 Work Stealing (Work_Stealing)

Work Stealing maintains one deque per CPU. New processes can be assigned using one of three strategies:

- `shortest_queue`: select the CPU with the fewest queued processes.
- `least_load`: select the CPU with the lowest queued remaining time.
- `power_of_two`: randomly select two CPUs and use the shorter queue.

When a CPU has no local work, it selects the CPU with the longest deque and steals a process from the back of that deque. A stolen process receives the configured migration overhead.

Main characteristics:

- Distributed per-CPU queues.
- Idle CPUs initiate balancing.
- Migration occurs only when a CPU needs work.
- Local execution can reduce unnecessary movement.
- Stealing improves utilization for irregular workloads.
- Victim selection is based on queue length rather than a complete cost model.

2.2.6 Dynamic Load Balancing (LoadBalancing)

The Load Balancing scheduler maintains one FIFO queue per CPU and combines proactive push migration with idle-CPU pull migration.

New processes are currently distributed in round-robin order:

```
target_cpu = next_process_index % num_cpu
```

The scheduler then continuously evaluates CPU load:

```
CPU load = sum of remaining time of all processes in that CPU
           queue
```

Push migration:

1. Find the busiest CPU.
2. Find the least-loaded CPU.
3. Check whether their load difference exceeds the threshold.
4. Select a waiting process from the busiest CPU.
5. Migrate it only if the predicted load difference becomes smaller.

Pull migration:

1. Detect an idle CPU.
2. Find a busy donor CPU with waiting work.
3. Move a waiting process if the migration improves balance.

The running process is never migrated. Migration candidates are selected from waiting processes, preferring the process with the greatest remaining time. Migration overhead is added to the process's remaining time.

Strengths:

- Corrects load imbalance after initial assignment.
- Combines proactive and idle-driven migration.
- Avoids migrating the currently running process.
- Rejects migrations that do not improve the load difference.
- Records migration events and detailed queue history.

Limitations:

- Tick-by-tick simulation can be expensive for very large burst times.
- Round-robin initial placement does not use current load.
- Migration cost is represented only as additional execution time.
- CPU cache, memory locality, NUMA topology, and communication cost are not modeled.
- Priority is not used.

2.3 Program Architecture

The project is divided into three main layers.

2.3.1 Domain Model

`process/process.py` defines:

- **Process**: one process and its validated properties.
- **Processes**: a collection that assigns IDs, limits the process count, copies workloads, and provides sorted views.
- Input-related exceptions.

2.3.2 Scheduling Layer

The `algorithms` package defines:

- **Schedule**: abstract base class and shared metric/history support.
- **ScheduleStep**: one continuous execution interval.
- The six scheduler implementations.

2.3.3 Presentation Layer

`app.py` defines the Streamlit interface and Plotly visualizations. It converts table rows into domain objects, executes all schedulers, computes comparison data, and renders static and dynamic results.

2.4 Input Data

The program accepts process data through a form or editable table.

Input	Validation
Process ID	Required text label; duplicate IDs are rejected
Arrival Time	Integer greater than or equal to 0
Burst Time	Integer greater than 0
Priority	Integer greater than 0

Scheduler configuration includes:

Parameter	Purpose
CPU Count	Number of simulated processors
GLB_RR Quantum	Time slice for Global Round Robin
CPU Affinity Quantum	Time slice for CPU Affinity
Work Stealing Quantum	Time slice for Work Stealing
Work Stealing Strategy	Initial local-queue placement strategy
Load Balancing Threshold	Minimum load difference before push migration
Migration Overhead	Additional execution time caused by migration

The dashboard begins with a default workload of 20 processes designed to create uneven CPU loads and observable migration behavior.

2.5 Output Data

The program produces:

- A Gantt-style execution timeline.
- CPU utilization.
- Throughput.
- Total completion time.
- Migration event count.
- A comparison table for all schedulers.
- A time-frame selector for inspecting a specific state.
- Per-CPU running process information.
- Global or local waiting queues.
- An animated queue replay.

The shared metric formulas are:

Total time = maximum ScheduleStep end time

Busy time = sum of all execution interval lengths

CPU utilization = $\frac{\text{busy time}}{\text{number of CPUs} \times \text{total time}}$

Throughput = $\frac{\text{completed process count}}{\text{total time}}$

Turnaround time = completion time – arrival time

Waiting time = turnaround time – actual CPU execution time

Response time = first execution time – arrival time

2.6 Important Functions and Their Purposes

2.6.1 Process Functions and Classes

Component	Purpose
<code>Process._validate()</code>	Rejects invalid burst, arrival, and priority values
<code>Process.copy()</code>	Creates an independent simulation copy
<code>Processes.add()</code>	Adds a process and assigns an ID
<code>Processes.copy()</code>	Copies the complete workload
<code>Processes.sorted_by_arrival()</code>	Returns processes ordered for simulation

2.6.2 Shared Scheduler Functions

Component	Purpose
<code>Schedule._copy_sorted_processes()</code>	Copies and sorts input without mutating the original workload
<code>Schedule._reset_history()</code>	Clears history before a new simulation
<code>Schedule._record_history()</code>	Stores running and queue state
<code>Schedule._update_basic_metrics()</code>	Calculates utilization, throughput, turnaround, waiting, and response metrics

2.6.3 Global FIFO Functions (GLB_FIFO)

Function	Purpose
<code>__init__()</code>	Stores the CPU count and marks every CPU as idle
<code>is_cpu_available()</code>	Returns the first idle CPU together with its availability flag
<code>estimate()</code>	Runs the global FIFO simulation: enqueues arrivals into one shared queue, dispatches the front process to any idle CPU, runs it to completion, and records steps, history, and metrics

2.6.4 Global Round Robin Functions (GLB_RR)

Function	Purpose
<code>__init__()</code>	Stores the time quantum and CPU count and marks every CPU as idle
<code>is_cpu_available()</code>	Returns the first idle CPU together with its availability flag
<code>estimate()</code>	Runs the global round-robin simulation: each dispatch executes for at most one quantum, and an unfinished process returns to the back of the shared queue

2.6.5 Partitioned FIFO Functions (PAR_FIFO)

Function	Purpose
<code>__init__()</code>	Stores the CPU count, marks every CPU as idle, and flags the queue scope as local
<code>is_cpu_available()</code>	Returns the first idle CPU together with its availability flag
<code>cpu_load()</code>	Local helper that sums the remaining time of the running and queued processes on one CPU
<code>estimate()</code>	Runs the partitioned FIFO simulation: assigns each arrival to the least-loaded local queue and runs each process to completion without later migration

2.6.6 CPU Affinity Functions (CPU_Affinity)

Function	Purpose
<code>__init__()</code>	Stores the time quantum, CPU count, and hard/soft affinity flag and marks every CPU as idle
<code>is_cpu_available()</code>	Returns the first idle CPU together with its availability flag
<code>estimate()</code>	Runs the affinity simulation: binds each process to a CPU on first execution, keeps it on that CPU under hard affinity, and time-slices execution by the quantum

2.6.7 Work Stealing Functions (Work_Stealing)

Function	Purpose
<code>__init__()</code>	Stores the quantum, CPU count, placement strategy, and migration overhead, and creates one deque per CPU plus the process-to-CPU mapping
<code>is_cpu_available()</code>	Returns the first idle CPU together with its availability flag
<code>_place_task()</code>	Places a new arrival into a local deque using the selected strategy (<code>shortest_queue</code> , <code>least_load</code> , or <code>power_of_two</code>)
<code>estimate()</code>	Runs the work-stealing simulation: an idle CPU pulls from its own deque or steals from the back of the busiest deque, adding migration overhead to stolen processes

2.6.8 Load Balancing Functions (LoadBalancing)

Function	Purpose
<code>_reset()</code>	Clears schedule steps, CPU queues, history, and migration events
<code>_cpu_load()</code>	Calculates the total remaining workload assigned to one CPU
<code>_least_loaded_cpu()</code>	Selects the CPU with the smallest remaining workload
<code>_busiest_cpu()</code>	Selects the busiest CPU while excluding previously rejected candidates
<code>_append_step()</code>	Records execution and merges adjacent intervals of the same process
<code>_record_state()</code>	Records the running processes, waiting queues, and CPU loads
<code>_assign_new_arrivals()</code>	Distributes newly arrived processes among CPUs using round-robin placement
<code>_try_push_migration()</code>	Selects the largest eligible waiting process and migrates it when doing so reduces the load imbalance
<code>push_migration()</code>	Repeatedly pushes waiting processes from overloaded CPUs to the least-loaded CPU
<code>estimate()</code>	Runs the complete tick-based load-balancing simulation and calculates the resulting metrics

2.7 Testing

`test.py` contains regression checks for:

- Protection against input mutation.
- Correct Partitioned FIFO workload accounting.
- Empty queues after completion.
- Separation of running processes from waiting queues.
- History availability for all schedulers.
- Final idle snapshots.

The application was also exercised through Streamlit’s application testing interface to verify that each scheduler can be selected without producing UI exceptions.

3 Performance Evaluation

To compare the algorithms quantitatively, every scheduler was run on all 22 benchmark process sets defined in `Process_options.py`, using the default dashboard configuration (4 CPUs, GLB_RR quantum 15, CPU Affinity quantum 5, Work Stealing quantum 5 with

the `shortest_queue` strategy, and migration overhead 1). For each metric, the value reported below is the mean across all 22 process sets, taken from the `Mean` column of the corresponding result file (`turnaround_time.csv`, `waiting_time.csv`, `response_time.csv`, and `cpu_utilization.csv`).

Algorithm	Avg Turnaround	Avg Waiting	Avg Response	CPU Utilization
GLB_FIFO	83.02	54.64	54.64	87.06%
GLB_RR	93.72	65.34	19.83	90.94%
PAR_FIFO	83.02	54.64	54.64	87.06%
CPU_Affinity	103.79	75.42	7.66	72.08%
Work_Stealing	94.09	65.52	7.92	90.10%
Load Balancing	82.29	53.56	53.56	86.42%

The averages reveal the trade-offs between the policies. Load Balancing and the FIFO-based schedulers (GLB_FIFO and PAR_FIFO) achieve the lowest turnaround and waiting times, since their non-preemptive execution avoids repeatedly rotating long jobs. The preemptive schedulers (GLB_RR, CPU_Affinity, and Work_Stealing) achieve far lower response times because every process begins executing after at most one quantum, at the cost of higher turnaround. GLB_RR and Work_Stealing reach the highest CPU utilization, while CPU_Affinity has the lowest, reflecting the idle time that strict affinity can introduce when runnable work exists only for another CPU.

4 Program Interface

The graphical interface is implemented with Streamlit.

4.1 Process Management

The process management area provides:

- A form for adding or updating one process.
- A selector for removing a process.
- An editable process table.
- Validation messages for invalid or duplicate data.

The table is stored in Streamlit session state so it remains available when the page reruns after user interaction.

4.2 Scheduler Configuration

The sidebar contains sliders and selectors for CPU count, time quanta, work-stealing strategy, load-balancing threshold, and migration overhead.

Changing a setting causes all schedulers to run again using the updated configuration.

4.3 Static Evaluation

The user selects an algorithm using a segmented control. The interface then displays:

- A Gantt chart showing process execution on each CPU.
- CPU utilization.
- Throughput.
- Migration count.
- A comparison table containing all algorithms.

Migrated process intervals are marked with an asterisk after migration occurs.

4.4 Dynamic Visualization

The dynamic view contains:

- A time slider.
- The process currently running on each CPU.
- Waiting processes.
- Migration messages.
- Play and Pause controls for animation.

The visualization layout depends on queue organization:

- Global-queue algorithms show one shared waiting queue.
- Local-queue algorithms show a separate queue for each CPU.

This interface makes scheduling decisions observable instead of presenting only final numerical metrics.

5 Contribution

The project was done by all of the team members. The detailed contributions of each member have been clarified in the table below:

Full Name	Student ID	Contribution
<i>Pham Chau Long</i>	<i>202416714</i>	Load Balancing, UI, visualization, testing
<i>Tran Quang Huy</i>	<i>202416706</i>	Work Stealing and report writing
<i>Tran Tat Dat</i>	<i>202416673</i>	Global RR, CPU Affinity, architecture, documentation
<i>Vu Hong Nam</i>	<i>202416728</i>	Global FIFO and Partitioned FIFO

6 References

1. The Kernel Development Community, “Scheduler Domains,” *The Linux Kernel Documentation* <https://docs.kernel.org/scheduler/sched-domains.html>
2. R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, “Multiprocessor Scheduling,” *Operating Systems: Three Easy Pieces*, Version 1.10. <https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched-multi.pdf>
3. R. D. Blumofe and C. E. Leiserson, “Scheduling Multithreaded Computations by Work Stealing,” *Journal of the ACM*, vol. 46, no. 5, pp. 720–748, 1999. <https://doi.org/10.1145/324133.324234>
4. Python Software Foundation, “collections — Container Datatypes: deque,” *Python Documentation*. <https://docs.python.org/3/library/collections.html#collections.deque>
5. Streamlit, “st.data_editor,” *Streamlit API Reference*. https://docs.streamlit.io/develop/api-reference/data/st.data_editor
6. Streamlit, “Session State,” *Streamlit API Reference*. https://docs.streamlit.io/develop/api-reference/caching-and-state/st.session_state
7. Plotly, “Gantt Charts in Python,” *Plotly Python Documentation*. <https://plotly.com/python/gantt/>
8. Plotly, “Animations in Python,” *Plotly Python Documentation*. <https://plotly.com/python/animations/>
9. Project source code and Git history, “Scheduling Algorithm in Multiprocessor Systems,” local repository, accessed June 19, 2026.